SISMOSAFE

ARCHITETTURA E PROGETTO DEI CALCOLATORI

prof. Nicola Mazzocca

a.a. 2024/2025

Matteo Cuomo M63001756

Antonio Tufo M63001727

IDEA

L’idea alla base del progetto SismoSafe è quella di realizzare un sistema in grado di rilevare vibrazioni sismiche. Quando una vibrazione viene rilevata, il sistema invia automaticamente un messaggio di allerta, notificando in tempo reale l’evento.

Progettazione

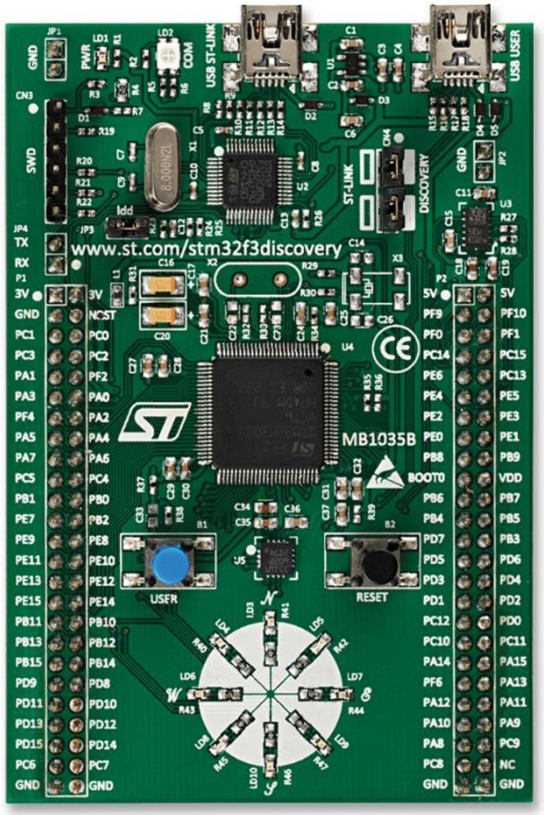
Nel nostro progetto sono presenti due nodi principali, ciascuno con un ruolo specifico:

- Nodo A si occupa dell’attivazione della vibrazione attraverso un motore DC. Riceve messaggi di tipo "STOP" dal Nodo B tramite comunicazione UART, e in risposta interrompe il funzionamento del motore.
- Nodo B monitora le vibrazioni tramite il sensore MPU6050. Se rileva un movimento anomalo e persistente, invia un messaggio "STOP" al Nodo A via UART e trasmette una notifica Telegram allo smartphone attraverso il modulo Wi-Fi ESP8266.

Componenti utilizzati

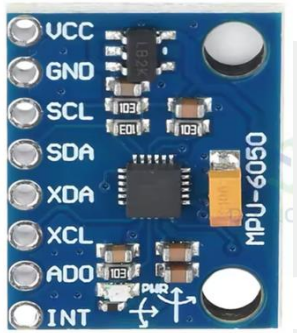
MOTORE DC

L298N



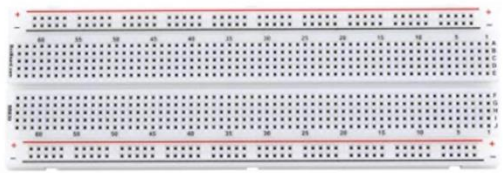
2x STM32F303- Discovery

MPU-6050

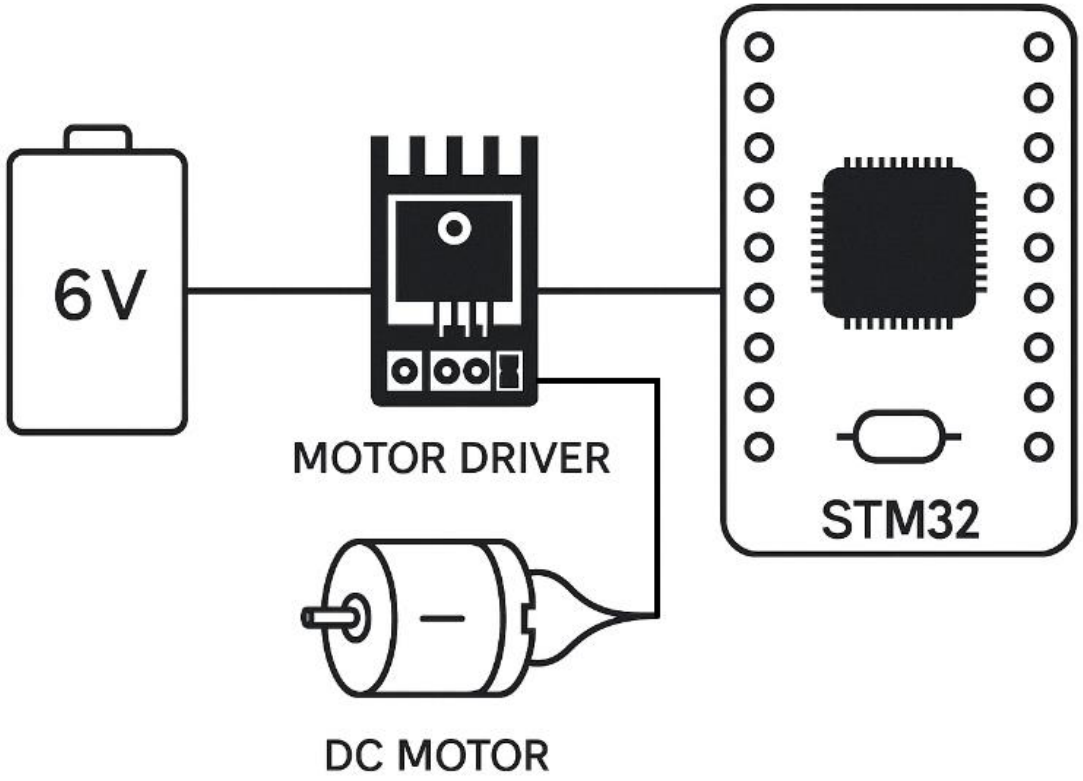


ESP-8266

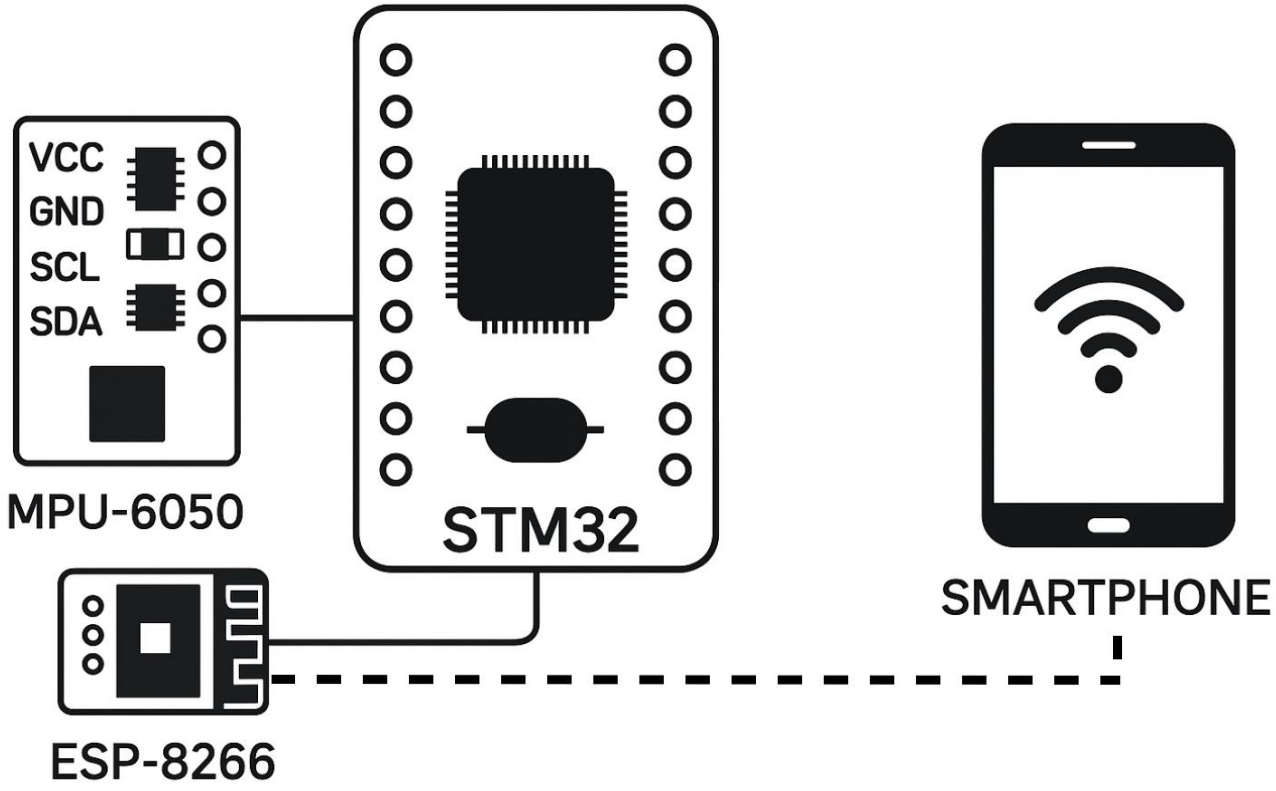
Altri componenti utilizzati…



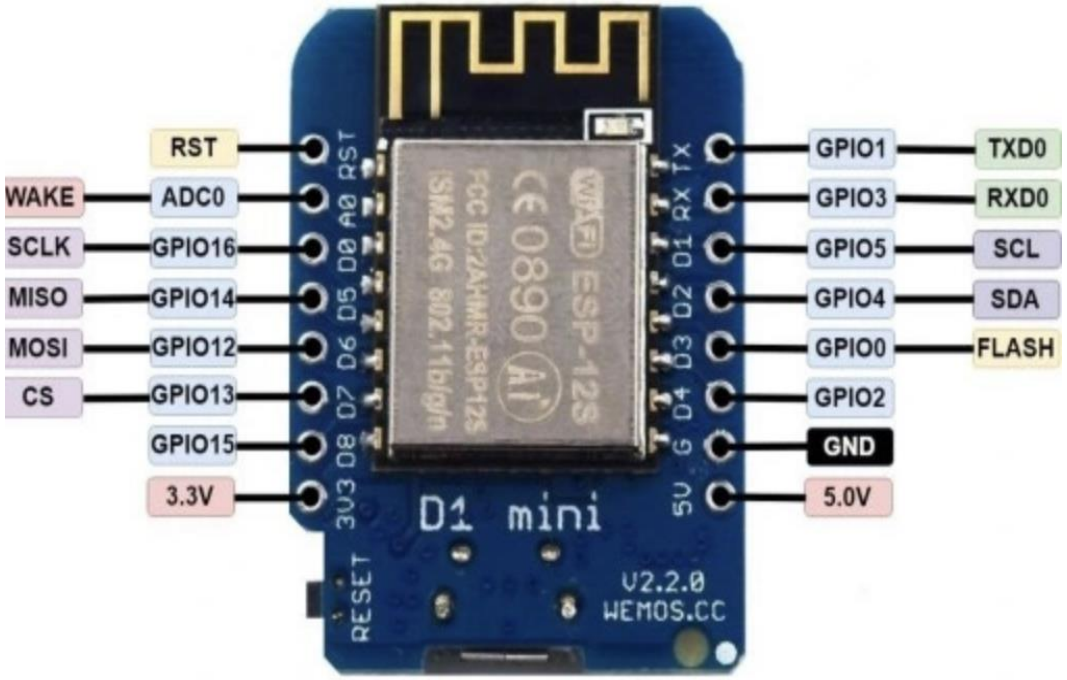
Schema di collegamento Nodo A



Schema di collegamento Nodo B



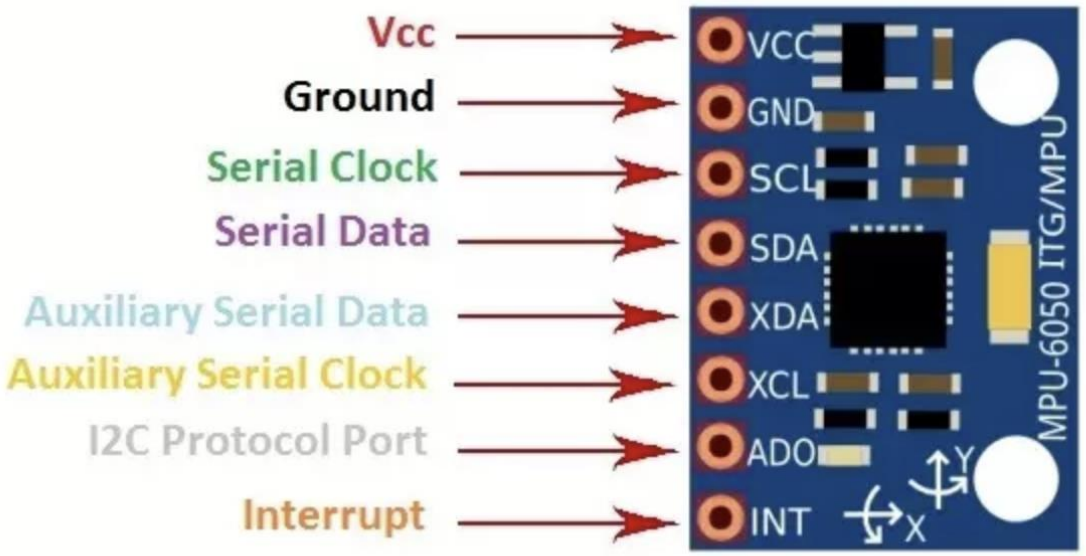
ESP-8266



L'ESP8266 è un modulo dotato di un processore a 32 bit che può eseguire codice in autonomia o in affiancamento a microcontrollori come STM32, comunicando tramite GPIO, UART, SPI, I2C, PWM, ADC. Integra una memoria RAM, una flash esterna per salvare il firmware, e supporta vari protocolli di comunicazione come HTTP o MQTT. Può funzionare sia come client, connettendosi a un router, sia come access point, creando una rete Wi-Fi propria. La sua versatilità e semplicità d'uso lo rendono ideale per inviare notifiche o controllare dispositivi da remoto via smartphone.

MPU-6050

L'MPU-6050 è un sensore che integra al suo interno un accelerometro a 3 assi e un giroscopio a 3 assi, permettendo di rilevare con precisione sia i movimenti lineari sia quelli rotazionali Questo sensore comunica tramite interfaccia I2C, rendendo semplice il collegamento con la maggior parte dei microcontrollori attraverso soli due fili: SCL (clock) e SDA (dati). Il modulo è alimentato tipicamente a 3.3V o 5V, a seconda della breakout board utilizzata, e fornisce dati continui sulle accelerazioni lungo gli assi X, Y e Z, oltre alla velocità angolare. Una caratteristica interessante dell'MPU-6050 è la presenza di una unità DMP (Digital Motion Processor) integrata, che consente l'elaborazione dei dati di movimento direttamente sul chip, riducendo il carico di calcolo sul microcontrollore principale. Questo è utile, ad esempio, per ottenere stime più stabili dell'orientamento o per filtrare automaticamente il rumore dei segnali.



Motore DC e Driver Motore

Il motore DC è un attuatore che trasforma l'energia elettrica in movimento rotatorio. Per farlo funzionare, è sufficiente applicare una tensione continua ai suoi terminali. Tuttavia, richiede una corrente superiore a quella che un microcontrollore (come l'STM32) può fornire, e non può essere controllato direttamente da esso.
Per questo entra in gioco il driver motore, come l'L298N, che agisce da intermediario: riceve i segnali logici dalla e fornisce al motore la corrente necessaria. In sintesi, il microcontrollore comanda, il driver esegue e il motore si muove:

- Senza il motore, non avremmo movimento.
- Senza il driver, il microcontrollore non sarebbe in grado di controllare il motore in modo sicuro ed efficace.

PROTOCOLLI DI COMUNICAZIONE

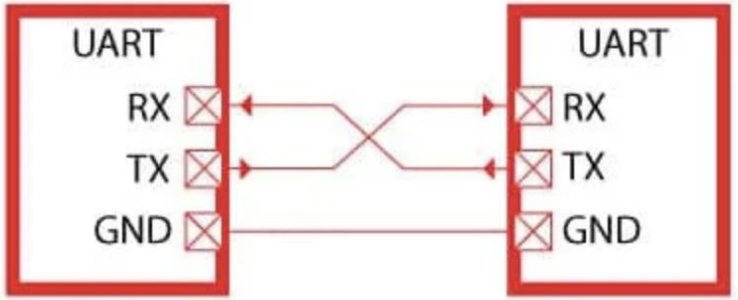
Nel nostro progetto, la comunicazione tra i componenti si fonda sull’utilizzo combinato dei protocolli UART e I2C, scelti per le loro caratteristiche complementari, perfettamente adatte ai ruoli funzionali richiesti dall’architettura distribuita del sistema.

- Il protocollo UART rappresenta il canale vitale di comunicazione interna ed esterna. Viene utilizzato in due direzioni fondamentali: da un lato, consente al Nodo B di trasmettere il messaggio di emergenza "STOP" al Nodo A; dall’altro, permette al Nodo B di dialogare con il modulo ESP8266, che invia una notifica Telegram allo smartphone. In questo modo, UART svolge il duplice ruolo di messaggero interno tra le unità del sistema e ponte verso l’esterno, per informare l’utente in tempo reale.
- Il protocollo I2C, invece, funge da interfaccia sensoriale. Attraverso di esso, il Nodo B comunica con il sensore MPU6050, raccogliendo continuamente dati sulle vibrazioni e i movimenti dell’ambiente circostante. Grazie all’I2C, il sistema è in grado di “percepire” ciò che accade e di reagire tempestivamente in caso di condizioni anomale.

UART

Per la comunicazione tra i nodi del sistema sono stati utilizzati i moduli USART integrati nelle schede STM32, configurati in modalità UART (Universal Asynchronous Receiver/Transmitter).
La UART è un protocollo di comunicazione seriale asincrona che consente lo scambio di dati tra due dispositivi elettronici utilizzando solo due fili principali: uno per la trasmissione (TX) e uno per la ricezione (RX).
Per stabilire il collegamento è sufficiente:

- collegare il TX di un dispositivo al RX dell'altro;
- il RX al TX;
- e mettere in comune il GND (massa) tra i due dispositivi.



I2C

L’I2C (Inter-Integrated Circuit) è un protocollo di comunicazione seriale sincrona progettato per consentire lo scambio di dati tra più dispositivi elettronici utilizzando solo due fili principali:

- SCL (Serial Clock Line): trasmette il segnale di clock generato dal master;
- SDA (Serial Data Line): trasporta i dati tra i dispositivi.

A queste vanno aggiunte altre due linee di collegamento ugualmente importanti, quali:

- Vcc: le linee SDA e SCL vi sono connesse mediante resistori di pull-up.
- Ground: definisce un riferimento comune per tutti i devices connessi.

È un protocollo master-slave: un solo dispositivo agisce come master (nel nostro caso l’STM32), mentre gli altri (come il sensore MPU6050) sono slave. Il master gestisce il clock e avvia ogni comunicazione.
Una delle principali caratteristiche dell’I2C è la sua capacità di collegare più dispositivi sullo stesso bus, grazie all’uso di indirizzi univoci per ciascuno. I dati vengono trasmessi in pacchetti (frame) composti da un indirizzo, un bit di lettura/scrittura, i dati stessi e un bit di ACK/NACK per confermare la ricezione. Il tutto è sincronizzato tramite il clock fornito dal master.

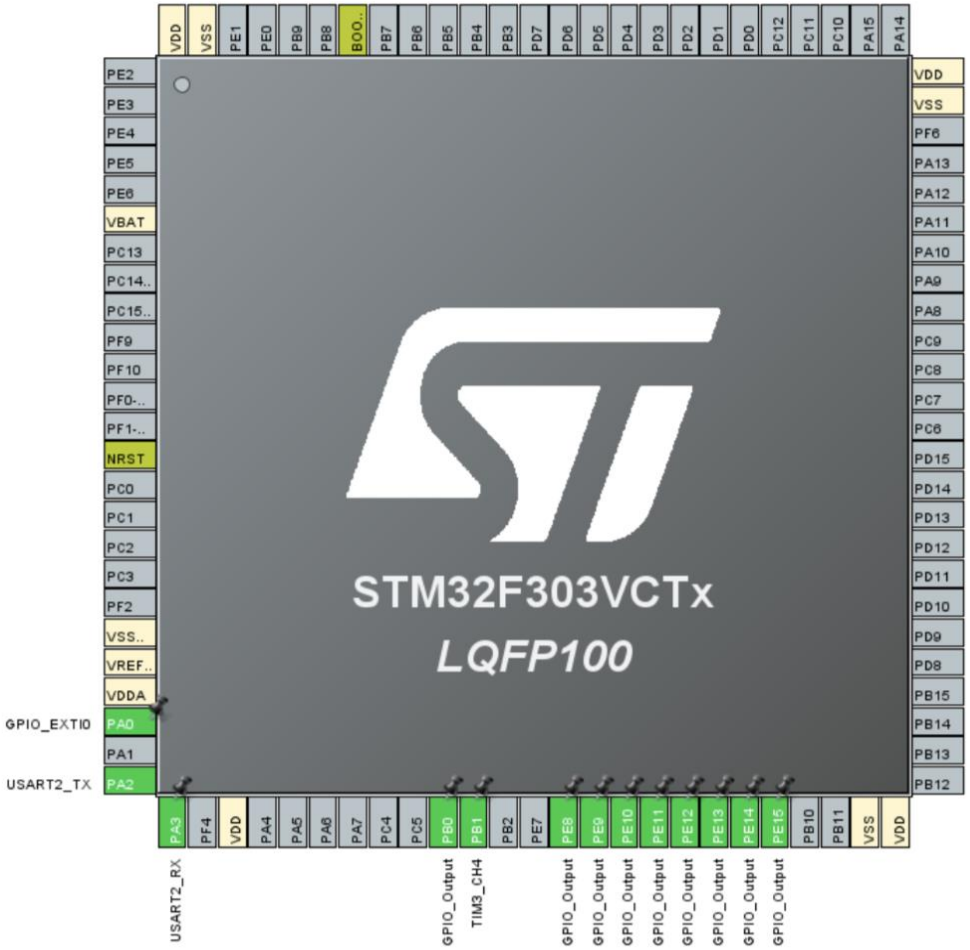
Codice stm32

Nel progetto *SismoSafe*, lo sviluppo del firmware delle board è stato realizzato utilizzando STM32CubeIDE, l'ambiente di sviluppo integrato ufficiale per microcontrollori STM32.
Per garantire una risposta tempestiva agli eventi esterni, è stato implementato il meccanismo delle interruzioni in due punti chiave del sistema del nodo A:

- Pulsante fisico (PA0): l'interruzione su fronte di salita consente di attivare o disattivare il motore senza dover controllare continuamente lo stato del pulsante nel main loop;
- Ricezione messaggio "STOP" via USART: l'interruzione sulla ricezione seriale permette di spegnere immediatamente il motore quando la board B rileva una vibrazione e invia il segnale di arresto.

In questo modo il microcontrollore non fa attesa attiva rispetto agli eventi cardine individuati in precedenza.

Board Nodo A



La board del nodo A presenta le seguenti configurazioni:

- Il pin PA0: (USER BUTTON) è configurato come GPIO con interruzione esterna (EXTI0). Alla pressione, viene attivata un'interruzione che consente di avviare o spegnere il motore DC.
- Il pin PA3: è configurato come USART2_RX per ricevere dati da Nodo B. Quando viene ricevuto il messaggio "STOP", un'interruzione viene attivata e il sistema spegne immediatamente il motore.
- Il pin PB1: è collegato all'ingresso ENA del driver del motore DC ed è utilizzato come uscita digitale per controllare la direzione o l'attivazione della rotazione.
- Il pin PB0: è collegato all'ingresso IN1 del driver ed è configurato per generare un segnale PWM tramite TIM3_CH4, utilizzato per regolare la velocità del motore.

Board A: Configurazione

NVIC Mode and Configuration

Mode

Configuration

NVIC | Code generation

Priority Group 4 bits ... | Sort by Premption Priority and Sub Priority | Sort by interrupts name

Search | Show available interrupts | Force DMA channels Int

NVIC Interrupt Table	Enabled	Preemption Priority	Sub
Non maskable interrupt	☑	0	0
Hard fault interrupt	☑	0	0
Memory management fault	☑	0	0
Pre-fetch fault, memory access fault	☑	0	0
Undefined instruction or illegal state	☑	0	0
System service call via SWI instruction	☑	0	0
Debug monitor	☑	0	0
Pendable request for system service	☑	0	0
Time base: System tick timer	☑	15	0
PVD interrupt through EXTI line16	☐	0	0
Flash global interrupt	☐	0	0
RCC global interrupt	☐	0	0
EXTI line0 interrupt	☑	0	0
TIM3 global interrupt	☐	0	0
USART2 global interrupt / USART2 wake-up interrupt through EXTI line ...	☑	0	0
Floating point unit interrupt	☐	0	0

TIM3 Mode and Configuration

Mode

Slave Mode: Disable
Trigger Source: Disable
Clock Source: Disable
Channel1: Disable
Channel2: Disable
Channel3: Disable
Channel4: PWM Generation CH4
Combined Channels: Disable

Configuration

Reset Configuration

User Constants | NVIC Settings | DMA Settings | GPIO Settings | Parameter Settings

Configure the below parameters :

Search (Ctrl+F)

Counter Settings	
Prescaler (PSC - 16 bits value)	71
Counter Mode	Up
Counter Period (AutoReload Register - ...	999
Internal Clock Division (CKD)	No Division
auto-reload preload	Enable
Trigger Output (TRGO) Parameters	
Master/Slave Mode (MSM bit)	Disable (Trigger input effect not delayed)
Trigger Event Selection TRGO	Reset (UG bit from TIMx_EGR)
Clear Input	
Clear Input Source	Disable
PWM Generation Channel 4	
Mode	PWM mode 1
Pulse (16 bits value)	500
Output compare preload	Enable
Fast Mode	Disable

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings | GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

Basic Parameters	
Baud Rate	9600 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1
Advanced Parameters	
Data Direction	Receive Only
Over Sampling	16 Samples
Single Sample	Disable

GPIO Mode and Configuration

Mode

Configuration

Group By Peripherals

GPIO | TIM | USART | NVIC

Search Signals

Search (Ctrl+F) | Show only Modified Pins

Pin Na...	Signal on...	GPIO out...	GPIO mode	GPIO Pull...	Maximu...	Fast Mode	User Label	Modified
PA0	n/a	n/a	External I...	No pull-...	n/a	n/a		☐
PE11	n/a	Low	Output P...	No pull-...	Low	n/a		☐
PE12	n/a	Low	Output P...	No pull-...	Low	n/a		☐
PE13	n/a	Low	Output P...	No pull-...	Low	n/a		☐
PE14	n/a	Low	Output P...	No pull-...	Low	n/a		☐
PE15	n/a	Low	Output P...	No pull-...	Low	n/a		☐

USART2 Mode and Configuration

Mode

Mode: Asynchronous
Hardware Flow Control (RS232): Disable
☐ Hardware Flow Control (RS485)

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings | GPIO Settings

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
USART2 global interrupt / USART2 wake-up interrupt through EXTI line 26	☑	0	0

```c
#include "main.h"
#include <string.h>

TIM_HandleTypeDef htim3;
UART_HandleTypeDef huart2;

uint8_t motor_state = 0;
uint8_t button_prev_state = 0;
uint8_t rx_buffer[10];

// Array dei pin LED PE8-PE15 (LD3-LD10)
GPIO_TypeDef* led_ports[] = {GPIOE, GPIOE, GPIOE, GPIOE,
uint16_t led_pins[] = {
  GPIO_PIN_8,  // LD4 (Blue)
  GPIO_PIN_9,  // LD3 (Red)
  GPIO_PIN_10, // LD5 (Orange)
  GPIO_PIN_11, // LD7 (Green)
  GPIO_PIN_12, // LD9 (Blue)
  GPIO_PIN_13, // LD10 (Red)
  GPIO_PIN_14, // LD8 (Orange)
  GPIO_PIN_15  // LD6 (Green)
};

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM3_Init(void);
static void MX_USART2_UART_Init(void);

void rotate_leds(void);
```

Board A: Main(1)

Dichiarazione variabili e dichiarazione delle funzioni

```c
int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_TIM3_Init();
  MX_USART2_UART_Init();

  HAL_UART_Receive_IT(&huart2, rx_buffer, 4);  // Ricevi "STOP"

  while (1)
  {
    uint8_t button_current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);

    if (button_current == GPIO_PIN_SET && button_prev_state == GPIO_PIN_RESET)
    {
      motor_state = !motor_state;

      if (motor_state)
      {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);    // IN1 HIGH
        HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_4);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, 500);     // Duty 50%
      }
      else
      {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);  // IN1 LOW
        HAL_TIM_PWM_Stop(&htim3, TIM_CHANNEL_4);
      }

      HAL_Delay(200);
    }

    button_prev_state = button_current;
  }
}
```

Board A: Main(2)

Inizializzazione e ciclo caldo

Board A: Main(3)

Interrupt di ricezione USART

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART2)
  {
    if (strncmp((char*)rx_buffer, "STOP", 4) == 0)
    {
      HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);
      HAL_TIM_PWM_Stop(&htim3, TIM_CHANNEL_4);
      motor_state = 0;

      // Accendi tutti i LED subito per visibilità iniziale
      for (int i = 0; i < 8; i++)
        HAL_GPIO_WritePin(led_ports[i], led_pins[i], GPIO_PIN_SET);
      HAL_Delay(200);
      for (int i = 0; i < 8; i++)
        HAL_GPIO_WritePin(led_ports[i], led_pins[i], GPIO_PIN_RESET);


      // Spegni tutti
      for (int i = 0; i < 8; i++)
        HAL_GPIO_WritePin(led_ports[i], led_pins[i], GPIO_PIN_RESET);
    }

    HAL_UART_Receive_IT(&huart2, rx_buffer, 4);
  }
}
```

Gestione dei conflitti nel nodo A(1)

Nel Nodo A del progetto sono presenti due sorgenti di interruzione potenzialmente in conflitto: il pulsante fisico collegato al pin PA0 (gestito tramite l'interrupt esterno EXTI0) e la ricezione di messaggi via USART2 (interrupt USART2_IRQn). Per garantire un comportamento corretto e prevedibile del sistema, è stata adottata una gestione attenta delle priorità e delle interazioni tra questi due eventi asincroni.
Innanzitutto, è stata assegnata una priorità più alta all'interrupt di ricezione UART rispetto a quella del pulsante. In particolare, nel main() viene eseguita la seguente configurazione:

```
HAL_NVIC_SetPriority(USART2_IRQn, 0, 0);   // priorità più alta
HAL_NVIC_SetPriority(EXTI0_IRQn, 1, 0);    // priorità più bassa
```

Questo garantisce che, in caso di concorrenza, il comando ricevuto via USART2 (ad esempio il messaggio "STOP") venga gestito con priorità rispetto a un'eventuale pressione del pulsante da parte dell'utente. In altre parole, l'ordine di arresto del motore ha precedenza su ogni altra azione locale, assicurando un comportamento affidabile e sicuro del sistema.

Gestione dei conflitti nel nodo A(2)

Inoltre, all'interno della callback HAL_UART_RxCpltCallback, in corrispondenza della ricezione del messaggio "STOP", l'interrupt del pulsante viene temporaneamente disabilitato tramite:

HAL_NVIC_DisableIRQ(EXTI0_IRQn);

Questa disattivazione impedisce che l'utente possa premere il pulsante durante la gestione della sequenza di arresto del motore e dell'animazione con i LED. Al termine della sequenza, l'interrupt viene riattivato con:

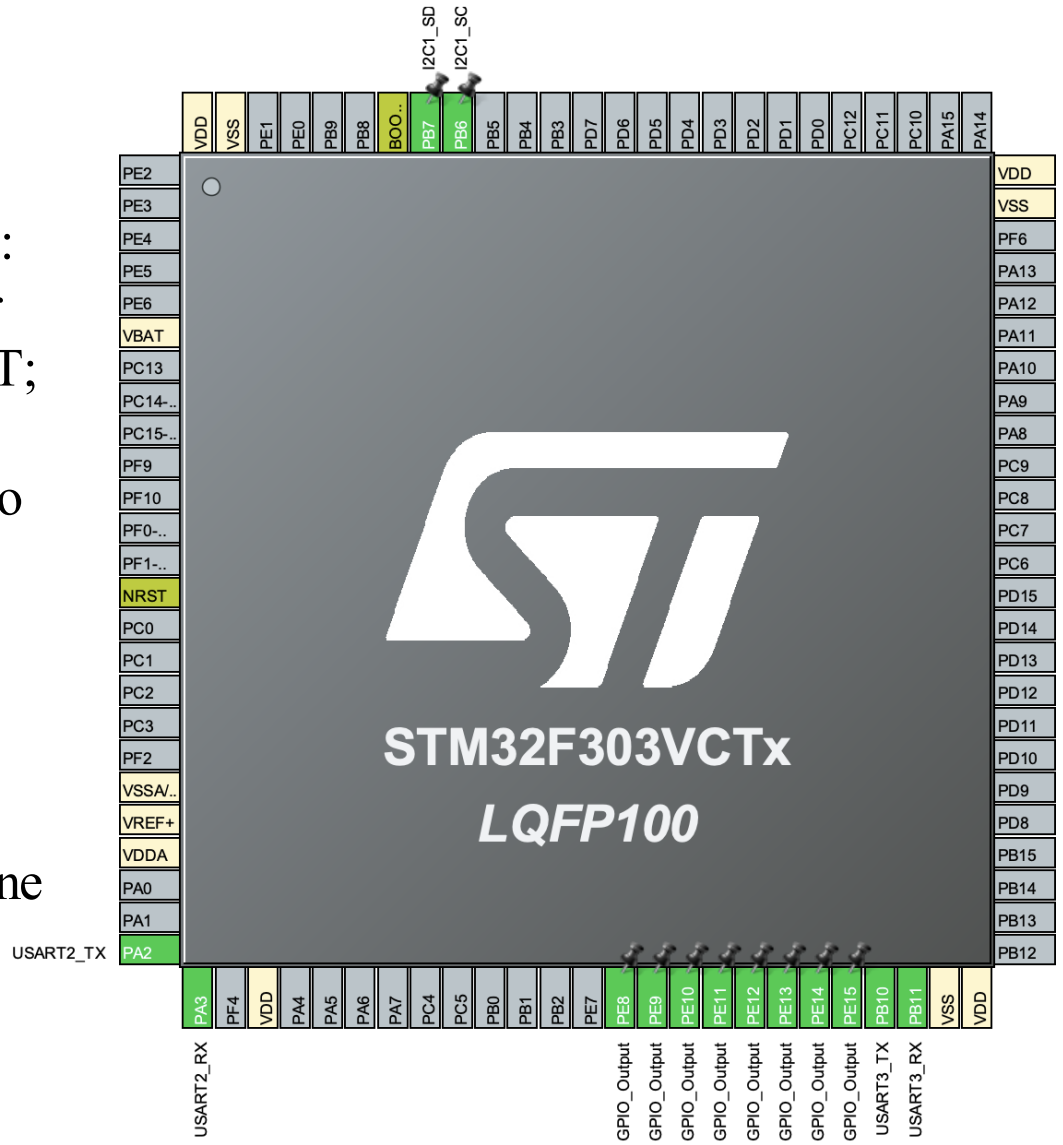
HAL_NVIC_EnableIRQ(EXTI0_IRQn);

Questo meccanismo garantisce che l'utente possa nuovamente utilizzare il pulsante solo dopo che il sistema ha completato la gestione del comando critico ricevuto via UART.

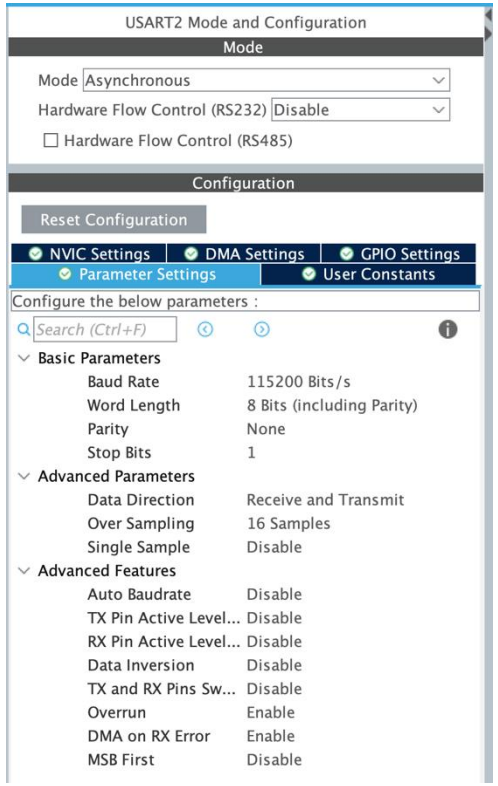
Board Nodo B

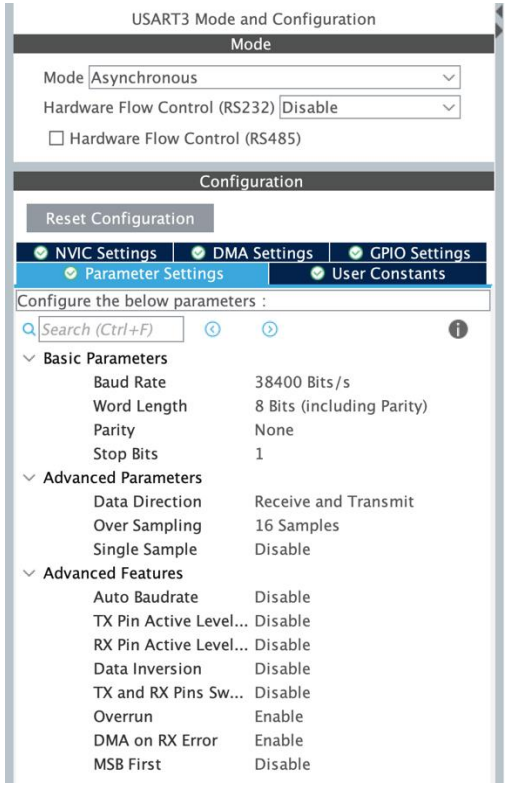
La board del nodo B presenta le seguenti configurazioni:

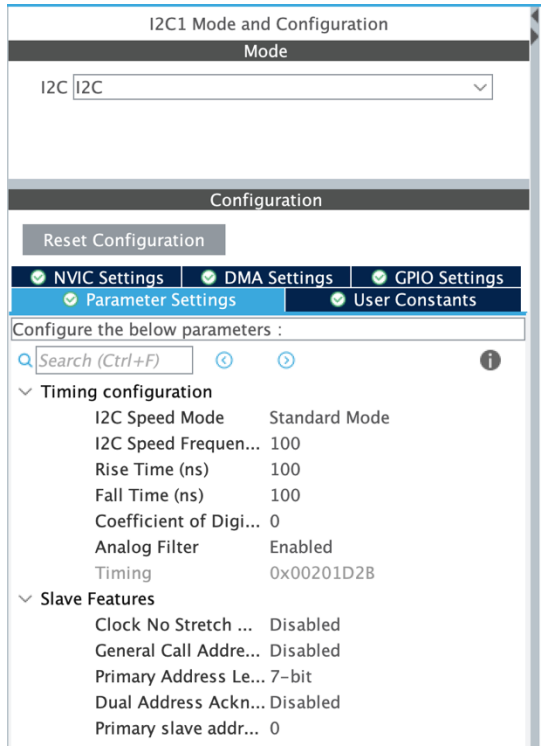
- PB10: configurato come USART3_TX, utilizzato per inviare il messaggio "STOP" alla Board A via USART;
- PB11 – USART3_RX *(opzionale):* Ricezione seriale, configurato ma non utilizzato attivamente nel progetto attuale;
- PA2: configurato come USART2_TX, utilizzato per inviare comandi o notifiche al modulo ESP8266;
- PA3: configurato come USART2_RX, utilizzato per ricevere eventuali risposte dal modulo ESP8266;
- PB6 – I2C1_SCL: Linea di clock per la comunicazione I2C con il sensore MPU6050;
- PB7 – I2C1_SDA: Linea dati per la comunicazione I2C con il sensore MPU6050.

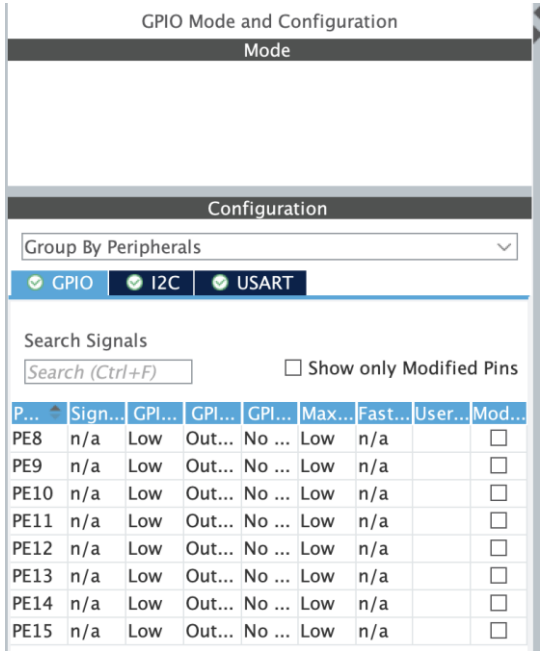


Board B: Configurazione









```c
#include "main.h"
#include <string.h>
#include <stdlib.h>
#include <stdio.h>

#define MPU_ADDR        (0x68 << 1)
#define ACC_X_H         0x3B
#define ACC_Y_H         0x3D
#define ACC_Z_H         0x3F
#define GYRO_X_H        0x43
#define GYRO_Y_H        0x45
#define GYRO_Z_H        0x47
#define PWR_MGMT_1      0x6B
#define THRESHOLD_ACC   8000
#define THRESHOLD_GYRO  500

I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef huart2;
UART_HandleTypeDef huart3;

// LED PE8-PE15
GPIO_TypeDef* led_ports[8] = {GPIOE, GPIOE, GPIOE, GPIOE, GPIOE, GP
uint16_t led_pins[8] = {
  GPIO_PIN_8, GPIO_PIN_9, GPIO_PIN_10, GPIO_PIN_11,
  GPIO_PIN_12, GPIO_PIN_13, GPIO_PIN_14, GPIO_PIN_15
};

uint8_t alarm_message[] = "ALARM\n";
uint8_t stop_message[]  = "STOP\n";

// Stato rotazione LED
uint8_t rotation_active = 0;
uint32_t rotation_start_time = 0;
uint8_t alarm_sent = 0;
```

Board B: Main(1)

Dichiarazione variabili

Board B: Main(2)

Inizializzazione delle periferiche e gestione del sensore

```c
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_USART3_UART_Init(void);

void MPU_Init(void);
int16_t Read_Axis(uint8_t reg_high);

void rotate_LEDs(void)
{
  static uint8_t index = 0;
  static uint32_t last_rotate_time = 0;

  if (HAL_GetTick() - last_rotate_time >= 150)
  {
    for (int i = 0; i < 8; i++)
    {
      HAL_GPIO_WritePin(led_ports[i], led_pins[i], (i == index) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    }
    index = (index + 1) % 8;
    last_rotate_time = HAL_GetTick();
  }
}

void turn_off_all_LEDs(void)
{
  for (int i = 0; i < 8; i++)
  {
    HAL_GPIO_WritePin(led_ports[i], led_pins[i], GPIO_PIN_RESET);
  }
}

void MPU_Init(void)
{
  uint8_t config[2] = {PWR_MGMT_1, 0x00};
  HAL_I2C_Master_Transmit(&hi2c1, MPU_ADDR, config, 2, HAL_MAX_DELAY);
}

int16_t Read_Axis(uint8_t reg_high)
{
  uint8_t data[2];
  HAL_I2C_Master_Transmit(&hi2c1, MPU_ADDR, &reg_high, 1, HAL_MAX_DELAY);
  HAL_I2C_Master_Receive(&hi2c1, MPU_ADDR, data, 2, HAL_MAX_DELAY);
  return (int16_t)((data[0] << 8) | data[1]);
}
```

Board B: Main(3)

Inizializzazione e ciclo caldo

```c
int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_I2C1_Init();
  MX_USART2_UART_Init();
  MX_USART3_UART_Init();
  MPU_Init();

  uint32_t vib_start = 0;
  uint8_t vib_confirmed = 0;
  uint8_t vib_active = 0;

  while (1)
  {
    int16_t acc_x = Read_Axis(ACC_X_H);
    int16_t acc_y = Read_Axis(ACC_Y_H);
    int16_t acc_z = Read_Axis(ACC_Z_H);
    int16_t gyro_x = Read_Axis(GYRO_X_H);
    int16_t gyro_y = Read_Axis(GYRO_Y_H);
    int16_t gyro_z = Read_Axis(GYRO_Z_H);

    int detected = (abs(acc_x) > THRESHOLD_ACC) || (abs(acc_y) > THRESHOLD_ACC) || (abs(acc_z - 16384) > THRESHOLD_ACC) ||
                   (abs(gyro_x) > THRESHOLD_GYRO) || (abs(gyro_y) > THRESHOLD_GYRO) || (abs(gyro_z) > THRESHOLD_GYRO);

    if (detected)
    {
      if (!vib_active)
      {
        vib_active = 1;
        vib_start = HAL_GetTick();
      }
      else if (!vib_confirmed && HAL_GetTick() - vib_start >= 300)
      {
        vib_confirmed = 1;
        rotation_active = 1;
        rotation_start_time = HAL_GetTick();
        HAL_UART_Transmit(&huart2, alarm_message, strlen((char *)alarm_message), HAL_MAX_DELAY);
        alarm_sent = 1;
      }
    }
    else if (!rotation_active)
    {
      vib_active = 0;
      vib_confirmed = 0;
    }


    if (rotation_active)
    {
      rotate_LEDs();

      if (HAL_GetTick() - rotation_start_time >= 20000)
      {
        rotation_active = 0;
        turn_off_all_LEDs();
        HAL_UART_Transmit(&huart3, stop_message, strlen((char *)stop_message), HAL_MAX_DELAY);
        HAL_Delay(100); // Attendi che il messaggio venga inviato
        NVIC_SystemReset(); // Riavvia la board
      }
    }

    if (!rotation_active && !vib_confirmed)
    {
      turn_off_all_LEDs();
    }

    HAL_Delay(20);
  }
}
```

Configurazione ESP-8266

Per semplificare l'integrazione del modulo Wi-Fi con il sistema SismoSafe, l'ESP8266 è stato programmato separatamente utilizzando Arduino IDE. Questa scelta consente di gestire in modo autonomo la connessione Wi-Fi e l'invio delle notifiche Telegram.

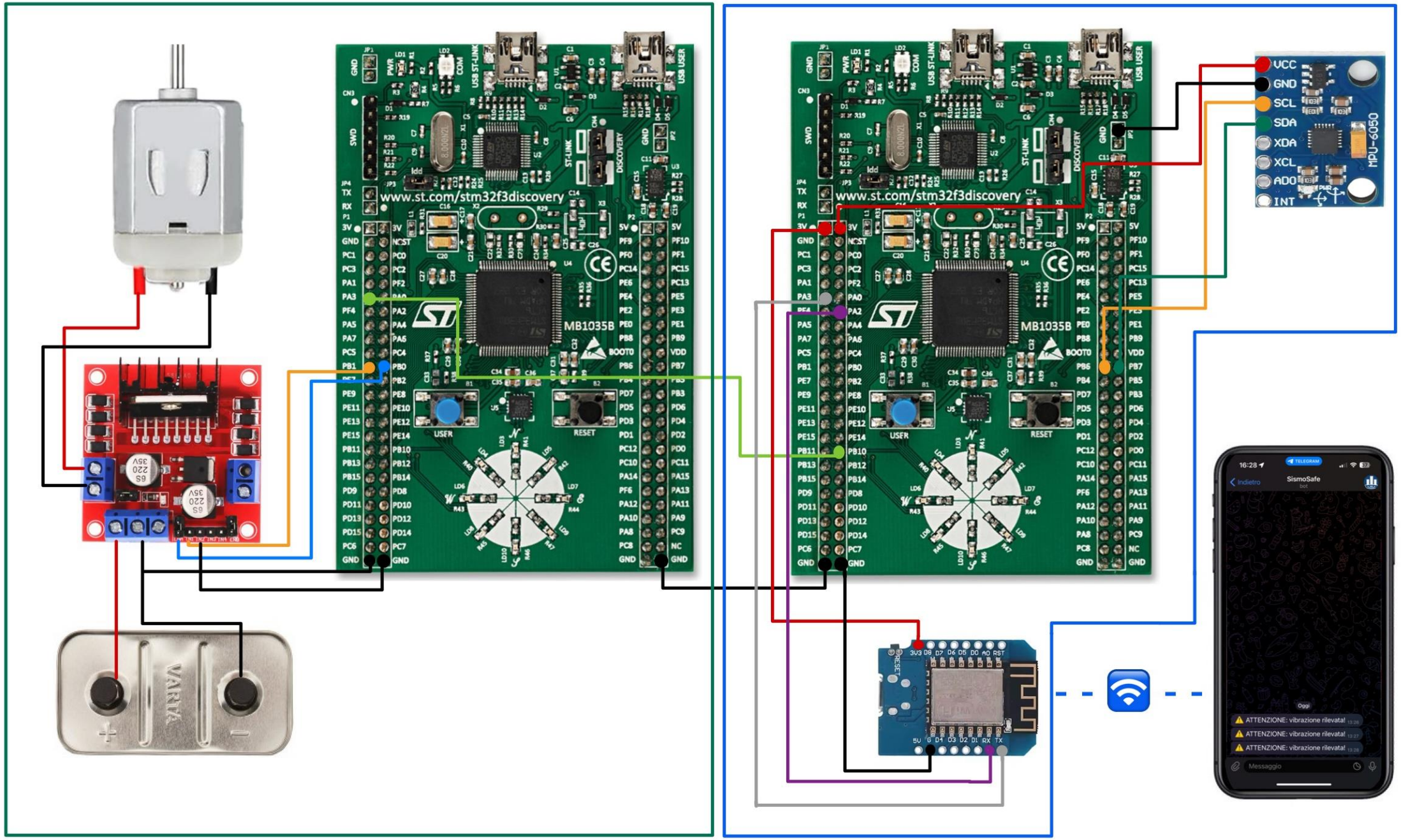
```
#include <ESP8266WiFi.h>
#include <WiFiClientSecure.h>

// Wi-Fi
const char* ssid     = "TIM-25618819";
const char* password = "$$DivanoLetto2025$$";

//const char* ssid     = "TP-Link_68E8";
//const char* password = "Arlo@2021";

// Telegram
const char* botToken = "7641123496:AAG0CmVaHueTREkqI3dRjwnLmedo19klk5g";
const char* chatID1  = "168282739";     // Primo telefono
const char* chatID2  = "501128439";     // Secondo telefono

const char* host = "api.telegram.org";
const int httpsPort = 443;

WiFiClientSecure client;
String serialBuffer = "";

void setup() {
  Serial.begin(115200);    // comunicazione da STM32 → ESP8266
  delay(1000);

  // Connessione Wi-Fi
  Serial.println("Connessione alla rete WiFi...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connesso!");

  client.setInsecure();  // accetta certificati SSL non verificati
}

void loop() {
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n') {
      serialBuffer.trim();  // rimuove spazi, \r, ecc.
      if (serialBuffer == "ALARM") {
        // Invia il messaggio a entrambi i telefoni
        sendTelegramMessage(chatID1, "⚠️ ATTENZIONE: vibrazione rilevata!");
        sendTelegramMessage(chatID2, "⚠️ ATTENZIONE: vibrazione rilevata!");
      }
      serialBuffer = "";  // reset buffer
    } else {
      serialBuffer += c;
    }
  }
}
```

Configurazione ESP-8266(2)

```
void sendTelegramMessage(const char* chatID, String message) {
  if (!client.connect(host, httpsPort)) {
    Serial.println("Connessione a Telegram fallita");
    return;
  }

  String url = "/bot" + String(botToken) + "/sendMessage?chat_id=" + String(chatID) + "&text=" + urlencode(message);
  Serial.print("Richiesta: ");
  Serial.println(url);

  client.print(String("GET ") + url + " HTTP/1.1\r\n" +
               "Host: " + host + "\r\n" +
               "Connection: close\r\n\r\n");

  // Leggi risposta (può servire per debug)
  while (client.connected()) {
    String line = client.readStringUntil('\n');
    if (line == "\r") break;
  }

  Serial.println("Notifica inviata a chatID: " + String(chatID));
}

// Codifica URL (per evitare errori con spazi e simboli)
String urlencode(String str) {
  String encodedString = "";
  char c;
  char code0, code1;
  for (int i = 0; i < str.length(); i++) {
    c = str.charAt(i);
    if (c == ' ') encodedString += '+';
    else if (isalnum(c)) encodedString += c;
    else {
      code1 = (c & 0xf) + '0';
      if ((c & 0xf) > 9) code1 = (c & 0xf) - 10 + 'A';
      c = (c >> 4) & 0xf;
      code0 = c + '0';
      if (c > 9) code0 = c - 10 + 'A';
      encodedString += '%';
      encodedString += code0;
      encodedString += code1;
    }
  }
  return encodedString;
}
```

NODO A

NODO B


www.st.com/stm32f3discovery
MB1035B
USER
RESET
VCC
GND
SCL
SDA
XDA
XCL
ADO
INT
MPU-6050
VARTA
SismoSafe
ATTENZIONE: vibrazione rilevata!
ATTENZIONE: vibrazione rilevata!
ATTENZIONE: vibrazione rilevata!